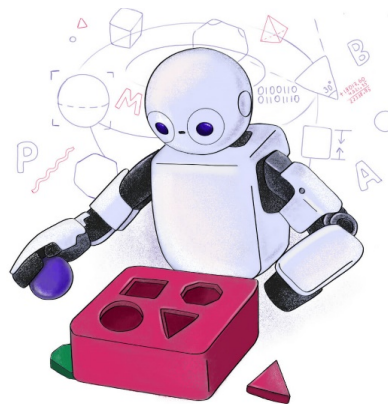


TP558 - Tópicos avançados em Machine Learning: *One-shot*



Inatel

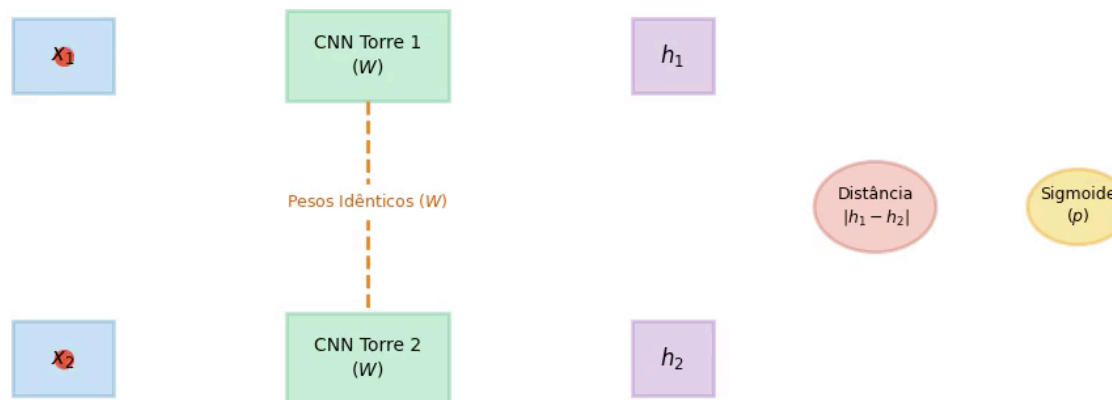
Fábio Augusto Pereira
fabio.pereira@mtel.inatel.br

Abertura

- O aprendizado de boas representações para aplicações de machine learning pode ser computacionalmente custoso e especialmente difícil quando há poucos dados disponíveis
- O cenário de one-shot learning é um exemplo de como o modelo deve fazer previsões corretas a partir de um único exemplo de cada nova classe
- Redes neurais siamesas porque empregam uma estrutura única para classificar naturalmente a similaridade entre entrada, que uma vez ajustada, a rede generaliza para novas classes de distribuições desconhecidas, sem necessidade de retreinamento

Redes Siamesas

Simulação do Fluxo de Informação - Rede Neural Siamesa



Etapa 1: Propagação dos dados de entrada nas redes convolucionais gêmeas.

Motivação

O Problema do Aprendizado com Poucos Dados

- Humanos são notavelmente capazes de reconhecer novos padrões a partir de pouquíssimos exemplos.
- Algoritmos de machine learning tradicionais, por outro lado, frequentemente falham quando há escassez de dados supervisionados.
- Existe um contraste fundamental: enquanto uma criança aprende a reconhecer um novo animal com apenas uma foto, redes convolucionais convencionais precisam de milhares de exemplos por classe para generalizar bem.

Zero-Shot Learning

Nenhum exemplo da classe alvo é observado antes da predição.

One-Shot Learning

Exatamente um exemplo por classe nova — foco deste trabalho.

Few-Shot Learning

Poucos exemplos (tipicamente 2–10) disponíveis por classe.

Trabalhos Relacionados: Breve Panorama

Fei-Fei et al. (2003, 2006)

Framework Bayesiano variacional para classificação one-shot. Classes previamente aprendidas ajudam a prever novas, com poucos exemplos disponíveis.

Lake et al. — HBPL (2013)

Hierarchical Bayesian Program Learning: modela o processo de desenho de caracteres generativamente. Decompõe imagens em partes estruturais, mas a inferência é intratável em grandes espaços de parâmetros.

Bromley & LeCun (1993)

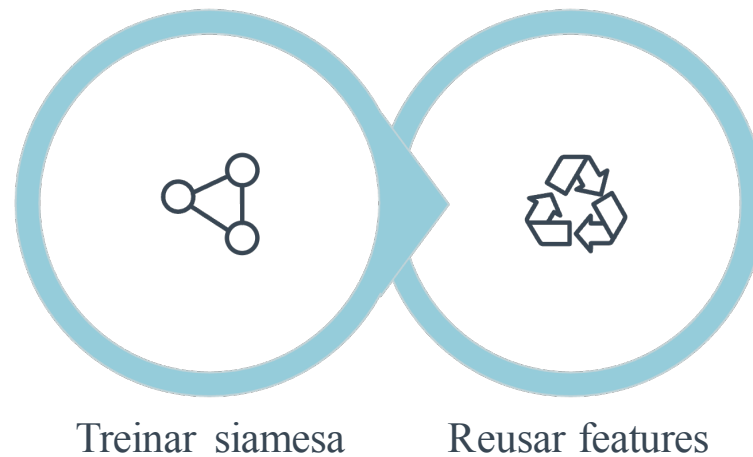
Introduziram redes siamesas para verificação de assinaturas como problema de casamento de imagens — precursores diretos da abordagem adotada neste artigo.

Outras Abordagens

Transfer learning (Lim, 2012), Redes de Markov ocultas hierárquicas para fala (Lake et al., 2014), e aprendizado por reforço com one-shot (Wu & Dennis, 2012).

Abordagem: Aprendizado Métrico Supervisionado

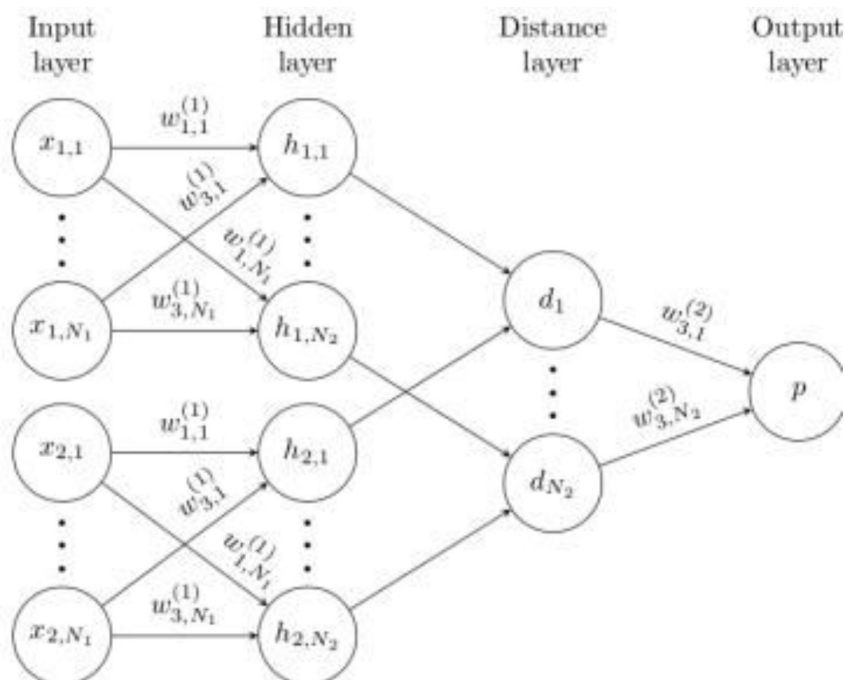
A estratégia central do trabalho pode ser resumida em duas etapas bem definidas:



Redes siamesas convolucionais são especialmente adequadas pois:

- aprendem features genéricas úteis para distribuições desconhecidas;
- são treinadas com técnicas padrão de otimização;
- não dependem de conhecimento de domínio específico, explorando apenas o framework de deep learning.

Redes Neurais Siamesas: Estrutura Fundamental



Propriedades Essenciais

Uma rede siamesa consiste em **duas redes gêmeas** que aceitam entradas distintas, mas são unidas por uma função de energia no topo que computa uma métrica entre as representações de mais alto nível.

- **Pesos amarrados (tied weights):** garantem que imagens similares sejam mapeadas para regiões próximas no espaço de features
- **Simetria:** a métrica é invariante à ordem dos inputs
- **Função de energia:** distância L_1 ponderada entre os vetores $\mathbf{h}_1$ e $\mathbf{h}_2$

Estratégia Geral

Treino por Verificação, Teste por One-Shot



Verification tasks (training)



One-shot tasks (test)

Fase de Treino — Verificação

O modelo é treinado com pares de imagens rotulados como **mesmo** ou **diferente**. Aprende a discriminar identidade de classes a partir de pares amostrados do conjunto de treino.

Fase de Teste — One-Shot

Para cada nova classe, apenas um exemplo é fornecido. A imagem de teste é comparada par a par com todos os candidatos. A classe com maior score de similaridade é a predição final.

Modelo: Arquitetura Convolutacional Siamesa

A rede possui L camadas, cada uma com N_l unidades. O vetor oculto da camada l para a primeira rede gêmea é $\mathbf{h}_{1,l}$ e para a segunda é $\mathbf{h}_{2,l}$. As primeiras $L - 2$ camadas usam ativação **ReLU**; as demais usam **sigmoidal**.

O k -ésimo mapa de features em cada camada convolutacional é calculado como:

$$a_{1,m}^{(k)} = \text{max-pool}(\max(0, \mathbf{W}_{l-1,l}^{(k)} * \mathbf{h}_{1,(l-1)} + \mathbf{b}_l), 2)$$

$$a_{2,m}^{(k)} = \text{max-pool}(\max(0, \mathbf{W}_{l-1,l}^{(k)} * \mathbf{h}_{2,(l-1)} + \mathbf{b}_l), 2)$$

onde $\mathbf{W}_{l-1,l}$ é o tensor 3D de mapas de features para a camada l e $*$ denota a operação de convolução válida. O stride fixo é 1, e o max-pooling usa filtro e stride de tamanho 2.

Predição Final: Métrica Induzida e Saída Sigmoidal

Após achatar as unidades da última camada convolucional em um único vetor, aplica-se uma camada totalmente conectada e então a **distância entre os gêmeos siameses**:

$$\mathbf{p} = \sigma \left(\sum_j \alpha_j \left| \mathbf{h}_{1,L-1}^{(j)} - \mathbf{h}_{2,L-1}^{(j)} \right| \right)$$

Interpretação

- σ : função sigmoideal, mapeia a saída para $[0, 1]$.
- α_j : pesos aprendidos que ponderam a distância componente a componente
- $\left| \mathbf{h}_1^{(j)} - \mathbf{h}_2^{(j)} \right|$: distância L_1 ponderada entre os vetores de features

Predição One-Shot

Para classificar uma imagem de teste $\mathbf{x}$ em C categorias, computa-se o score para cada par e prediz-se a classe de maior similaridade:

$$C = \operatorname{argmax}_c \mathbf{p}^{(c)}$$

Função de Custo: Cross-Entropy Regularizada

O treinamento utiliza um classificador binário com objetivo de **cross-entropy regularizada**. Seja M o tamanho do minibatch e $\mathbf{y}(x_1^{(i)}, x_2^{(i)})$ o vetor de rótulos — onde $y = 1$ para mesmo caractere e $y = 0$ para classes diferentes:

$$L(x_1^{(i)}, x_2^{(i)}) = \mathbf{y}(x_1^{(i)}, x_2^{(i)}) \log \mathbf{p}(x_1^{(i)}, x_2^{(i)}) + (1 - \mathbf{y}(x_1^{(i)}, x_2^{(i)})) \log (1 - \mathbf{p}(x_1^{(i)}, x_2^{(i)})) + \lambda^T \|\mathbf{w}\|^2$$

Termos da Função

- Primeiro termo: penaliza pares do mesmo par preditos como diferentes
- Segundo termo: penaliza pares distintos preditos como iguais
- Terceiro termo: regularização L_2 sobre os pesos $\mathbf{w}$

Backpropagation com Pesos Amarrados

O gradiente é **aditivo** entre as redes gêmeas devido ao compartilhamento de pesos. Isso simplifica a atualização e garante consistência na aprendizagem de features simétricas.

Otimização: Regra de Atualização com Momentum

O treinamento usa minibatches de tamanho 128 com taxa de aprendizado η_j , momentum μ_j e regularização L_2 com λ_j definidos por camada. A regra de atualização na época T é:

$$\mathbf{w}_{kj}^{(T)} = \mathbf{w}_{kj}^{(T)} + \Delta \mathbf{w}_{kj}^{(T)} + 2\lambda_j \|\mathbf{w}_{kj}\|$$
$$\Delta \mathbf{w}_{kj}^{(T)} = -\eta_j \nabla w_{kj}^{(T)} + \mu_j \Delta \mathbf{w}_{kj}^{(T-1)}$$

Agenda de Aprendizado

- Taxa de aprendizado decaída em 1% por época: $\eta_j^{(T)} = 0.99 \eta_j^{(T-1)}$
- Momentum iniciando em 0.5, aumentando linearmente até μ_j
- Máximo de 200 épocas; parada antecipada após 20 épocas sem melhora

Inicialização dos Pesos

- Camadas convolucionais: $N(0, 10^{-2})$
- Biases convolucionais: $N(0.5, 10^{-2})$
- Camadas FC: $N(0, 2 \times 10^{-1})$

Otimização de Hiperparâmetros

Foi utilizado o framework **Whetlab** (otimização Bayesiana) para seleção de hiperparâmetros, maximizando a acurácia de validação one-shot.



Taxa de Aprendizado

$\eta_j \in [10^{-4}, 10^{-1}]$ por camada



Momentum

$\mu_j \in [0, 1]$ por camada



Regularização L_2

$\lambda_j \in [0, 0.1]$ por camada



Filtros Convolucionais

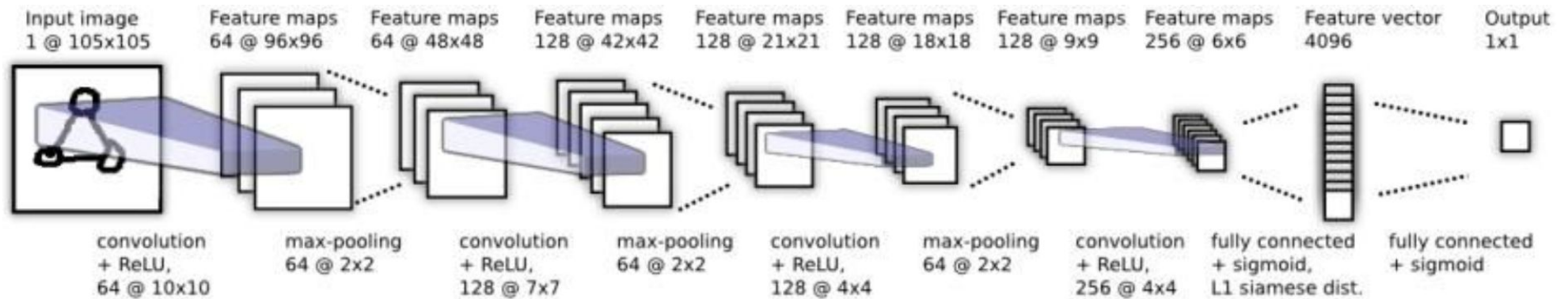
Tamanho: 3×3 a 20×20; quantidade: 16 a 256 (múltiplos de 16)



Camadas FC

128 a 4096 unidades (múltiplos de 16)

Melhor Arquitetura Convolutacional



1

Conv + ReLU
64 filtros 10x10, entrada 105x105

2

Max-Pool + Conv + ReLU
128 filtros 7x7

3

Max-Pool + Conv + ReLU
128 filtros 4x4

4

Conv + ReLU
256 filtros 4x4

5

FC + Sigmoid + Dist. L1
4096 unidades → saída 1x1

O Dataset Omniglot



Características do Dataset

- Coletado via Amazon Mechanical Turk por Brenden Lake (MIT)
- 50 alfabetos: de idiomas internacionais (Latino, Coreano) a fictícios (Klingon, Aurek-Besh)
- Imagens binárias de 105×105 pixels, desenhadas à mão
- 20 desenhistas por caractere

Divisão Experimental

- **Background set:** 40 alfabetos — treino e validação (tuning de hiperparâmetros)
- **Evaluation set:** 10 alfabetos — exclusivamente para medir one-shot performance

Tarefa de Classificação One-Shot: 20-Way



Como funciona a tarefa 20-way?

Um único caractere de teste deve ser identificado dentre 20 classes — cada uma representada por apenas um exemplo

- 20 classes distintas, uma imagem por classe;
- O modelo compara o teste com cada candidato;
- A classe com maior similaridade é predita;
- Dataset utilizado: Omniglot (caracteres de múltiplos alfabetos);

❑ Este cenário exige que o modelo generalize sem qualquer retreinamento — apenas com as features aprendidas previamente.

Protocolo de Avaliação One-Shot (20-way)

O protocolo de avaliação desenvolvido por Lake consiste em:

1 Seleção do Alfabeto

Um alfabeto é escolhido do conjunto de avaliação, junto com 20 caracteres sorteados aleatoriamente.

3 Comparação Par a Par

Cada imagem de teste é comparada individualmente contra todos os 20 caracteres do segundo desenhista, predizendo a classe de maior similaridade.

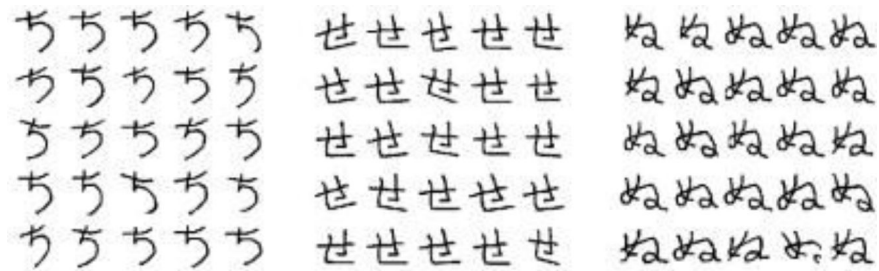
2 Geração de Imagens

Dois desenhistas produzem amostras dos 20 caracteres. Os caracteres do primeiro desenhista são as **imagens de teste**.

4 Repetição e Contagem

O processo é repetido duas vezes por alfabeto → 40 trials por alfabeto × 10 alfabetos = **400 trials no total**.

Aumento de Dados: Distorções Afins



Transformações Afins Estocásticas

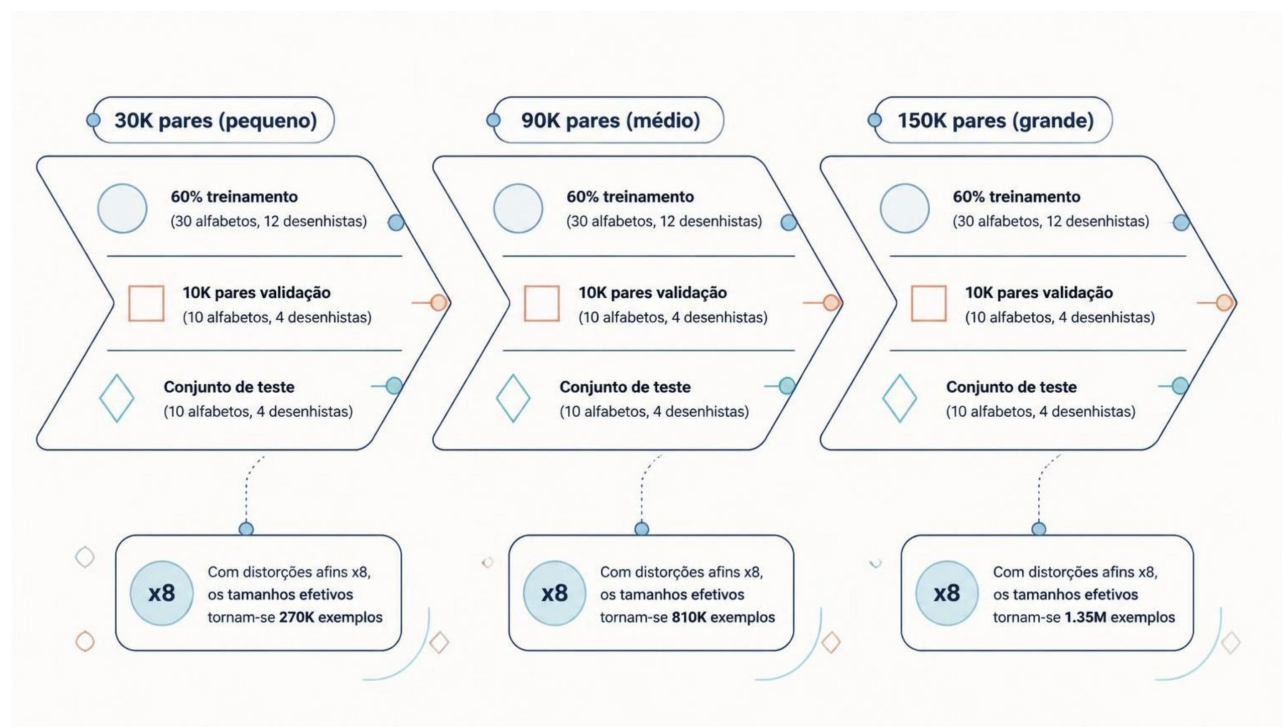
Para cada par de imagens $(\mathbf{x}_1, \mathbf{x}_2)$, são geradas transformações afins T_1, T_2 produzindo $\mathbf{x}'_1 = T_1(\mathbf{x}_1)$ e $\mathbf{x}'_2 = T_2(\mathbf{x}_2)$. Cada transformação T é parametrizada por:

$$T = (\theta, \rho_x, \rho_y, s_x, s_y, t_x, t_y)$$

- Rotação: $\theta \in [-10^\circ, 10^\circ]$
- Cisalhamento: $\rho_x, \rho_y \in [-0.3, 0.3]$
- Escala: $s_x, s_y \in [0.8, 1.2]$
- Translação: $t_x, t_y \in [-2, 2]$
- Cada componente incluída com probabilidade 0.5

Divisão dos Dados para Verificação

Para treinar a rede de verificação, foram criados três tamanhos de datasets amostrados com pares aleatórios **mesmo e diferente**.



Com a adição de 8 distorções afins por exemplo, os conjuntos efetivos chegam a 270K, 810K e 1.350K amostras, respectivamente, ampliando significativamente a diversidade do treino.

Resultados de Verificação

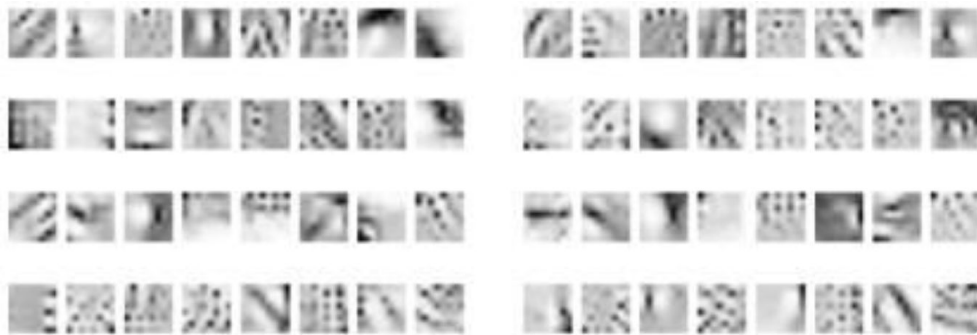
Método	Acurácia (%)
30k — sem distorções	90.61
30k — distorções afins ×8	91.90
90k — sem distorções	91.54
90k — distorções afins ×8	93.15
150k — sem distorções	91.63
150k — distorções afins ×8	93.42

Observações

- Distorções afins consistentemente melhoram a acurácia em todos os tamanhos de dataset
- O ganho marginal com mais dados diminui após 90k pares
- Melhor resultado: **93.42%** com 150k + distorções

📄 O critério de parada foi baseado no erro de validação one-shot (320 trials), ao menos tão eficaz quanto o erro de verificação.

Filtros Aprendidos na Primeira Camada



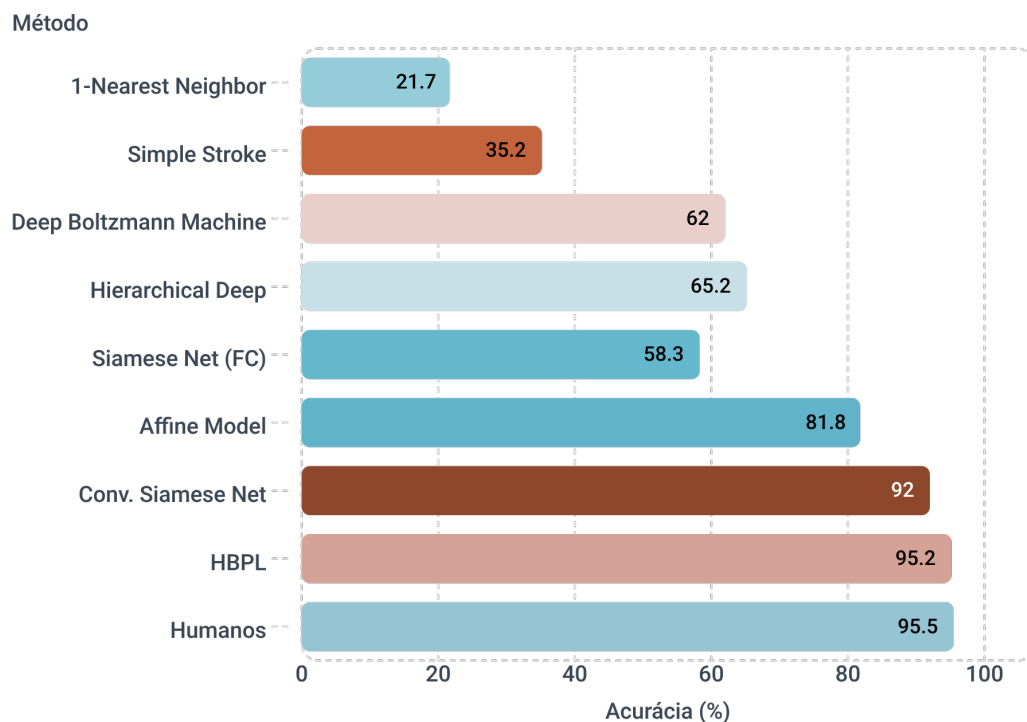
Papéis Distintos dos Filtros

Os 32 primeiros filtros extraídos das duas melhores redes (treinadas com 90k e 150k + distorções) revelam especialização espontânea:

- Alguns filtros detectam **características pontuais de pequena escala** (bordas finas, cantos)
- Outros funcionam como **detectores de bordas de grande escala** e texturas
- Há co-adaptação entre filtros, mas papéis claramente distintos emergem

Isso demonstra que a rede aprende uma representação rica e diversificada, sem supervisão sobre quais features extrair.

Resultados: One-Shot Learning no Omniglot



Análise Comparativa

- A **Rede Siamesa Convolucional** alcança **92%** de acurácia
- Supera todos os baselines exceto o HBPL (95.2%)
- HBPL incorpora conhecimento a priori sobre traços de escrita
- A rede siamesa **não utiliza nenhum conhecimento de domínio** sobre caracteres ou traços
- A diferença para o nível humano é de apenas 3.5 pontos percentuais

Generalização para MNIST: Teste One-Shot

Configuração do Experimento

O Omniglot é chamado de "MNIST transposto" — o número de classes supera o de instâncias. Para avaliar generalização, os 10 dígitos do MNIST foram tratados como um alfabeto em uma tarefa 10-way one-shot, sem nenhum fine-tuning no MNIST.

- 400 trials one-shot no conjunto de teste do MNIST
- Imagens 28×28 redimensionadas para 35×35
- Modelo treinado em Omniglot com imagens 35×35 (downsampled)

Método	Acurácia (%)
1-Nearest Neighbor	26.5
Conv. Siamese Net	70.3

📄 Queda de ~22pp em relação ao Omniglot, mas ainda **sem nenhum treino em MNIST** — evidência forte de transferência de features entre domínios.

Replicando Resultados

Trial one-shot 20-way — previsão correta (verde=acerto, vermelho=previsão errada, azul=classe correta)

Imagem de consulta (query)

g

p=0.04

p

p=0.30

d

p=0.23

d

p=0.01

b

p=0.51

?

p=0.72

g

p=0.97

g

p=0.02

g

p=0.12

g

p=0.26

g

p=0.01

g

p=0.22

g

p=0.02

g

p=0.26

g

p=0.02

g

p=0.00

g

p=0.00

g

p=0.04

g

p=0.05

g

p=0.34

g

Limitações

Desempenho

Depende da qualidade da representação aprendida

Classes

Se duas classes diferentes apresentam características visuais muito semelhantes, a rede pode produzir uma similaridade incorreta

Generalização

O experimento principal está concentrado no domínio de caracteres manuscritos, portanto a generalização para outros domínios precisa ser investigada (exemplo MNIST com 70,3%)

Arquitetura

Relativamente grande, quando considerado a camada totalmente conectada de 4096 unidades

Contribuições e Conclusões

Aprendizado Métrico Eficaz

Redes siamesas convolucionais aprendem uma métrica de similaridade poderosa via verificação, transferível a novas classes adequada para situações nas quais novas classes possuem poucos dados

Generalização sem Retreinamento

Features aprendidas generalizam para distribuições completamente novas — não precisa conhecer previamente as classes que serão avaliadas

Próximo ao Nível Humano

92% de acurácia no Omniglot indica que precisão humana é alcançável com aprendizado métrico baseado em deep learning

Sem Conhecimento de Domínio

Diferentemente do HBPL, a abordagem não depende de um modelo explícito sobre como os caracteres são desenhados

Resumo do Fluxo de Aprendizado



O pipeline combina engenharia cuidadosa de dados, arquitetura siamesa com pesos amarrados, otimização Bayesiana de hiperparâmetros e avaliação rigorosa — resultando em um sistema de one-shot learning robusto, geral e competitivo com o estado da arte.

1

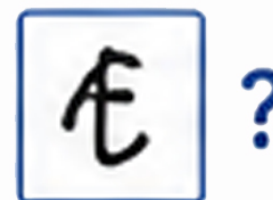
O PROBLEMA

Como reconhecer
novas classes
com apenas
um exemplo?

Classes conhecidas
(muitas amostras por classe)



Nova classe
(apenas 1 exemplo)



2

A IDEIA PRINCIPAL

Em vez de aprender “qual é a classe?”, a rede aprende
“quão semelhantes são duas imagens?”



Imagem A



Similaridade
(0 a 1)



Imagem B

3

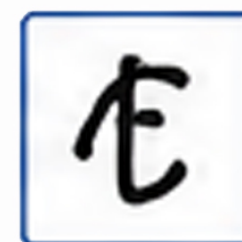
ARQUITETURA SIAMESA

Duas redes idênticas compartilham os mesmos pesos.

Imagem A

vetor h_1 Mesmos
pesosvetor h_2 

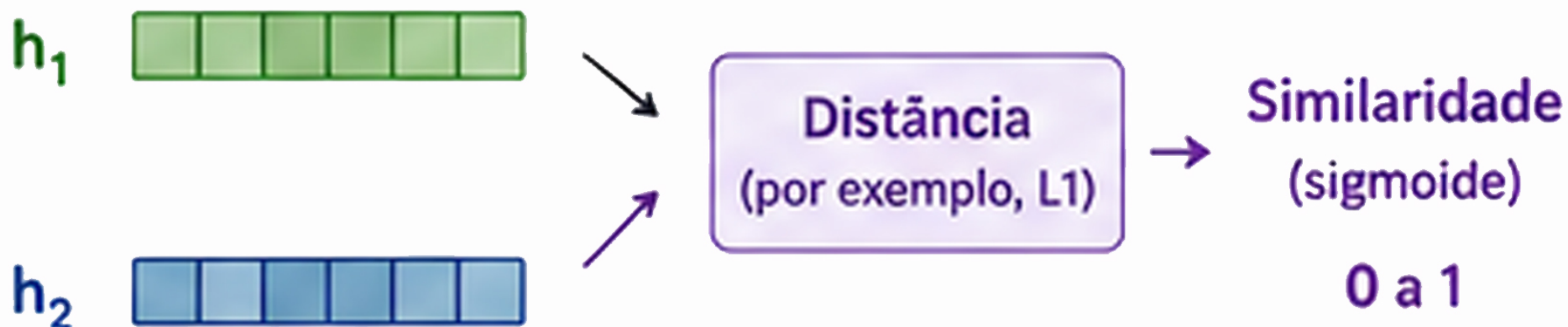
Imagem B



4

COMPARAÇÃO

Os vetores de características são comparados para produzir uma medida de similaridade.



5

TREINAMENTO (VERIFICAÇÃO)

A rede é treinada com pares rotulados como "mesma classe" (1) ou "classes diferentes" (0).



rótulo = 0
(classes diferentes)



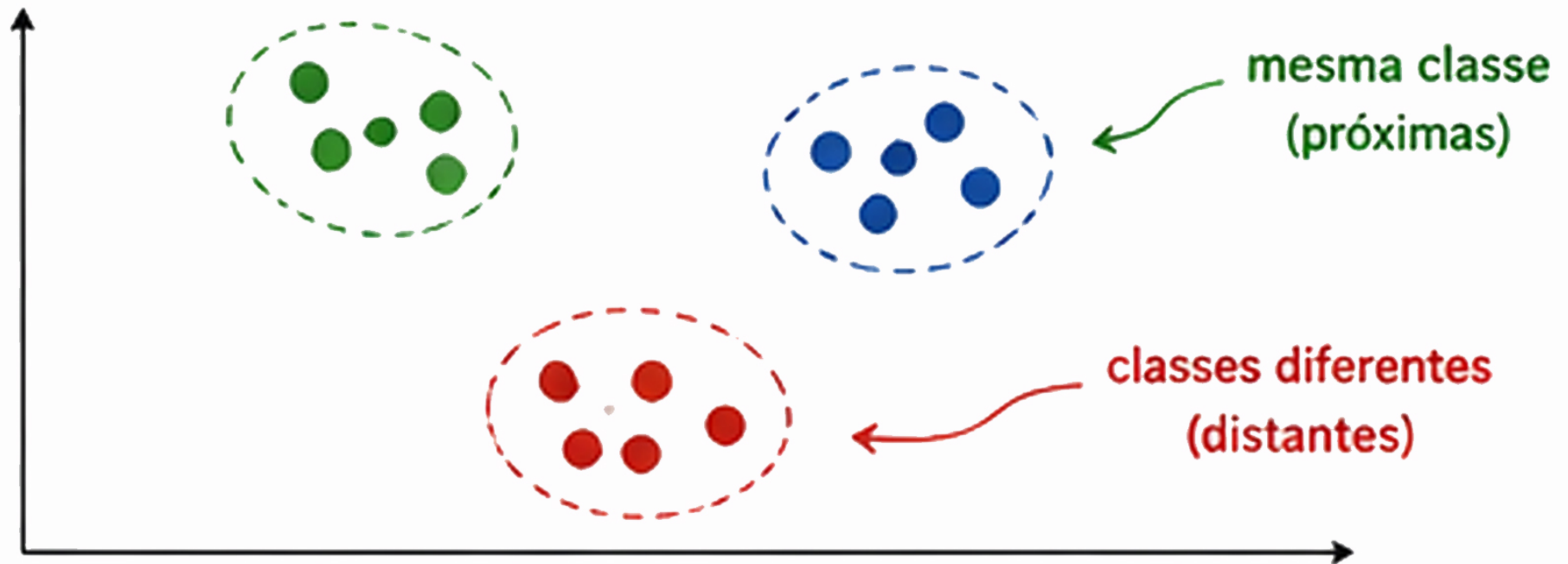
rótulo = 1
(mesma classe)

Perda: Entropia cruzada binária + regularização

6

APÓS O TREINAMENTO

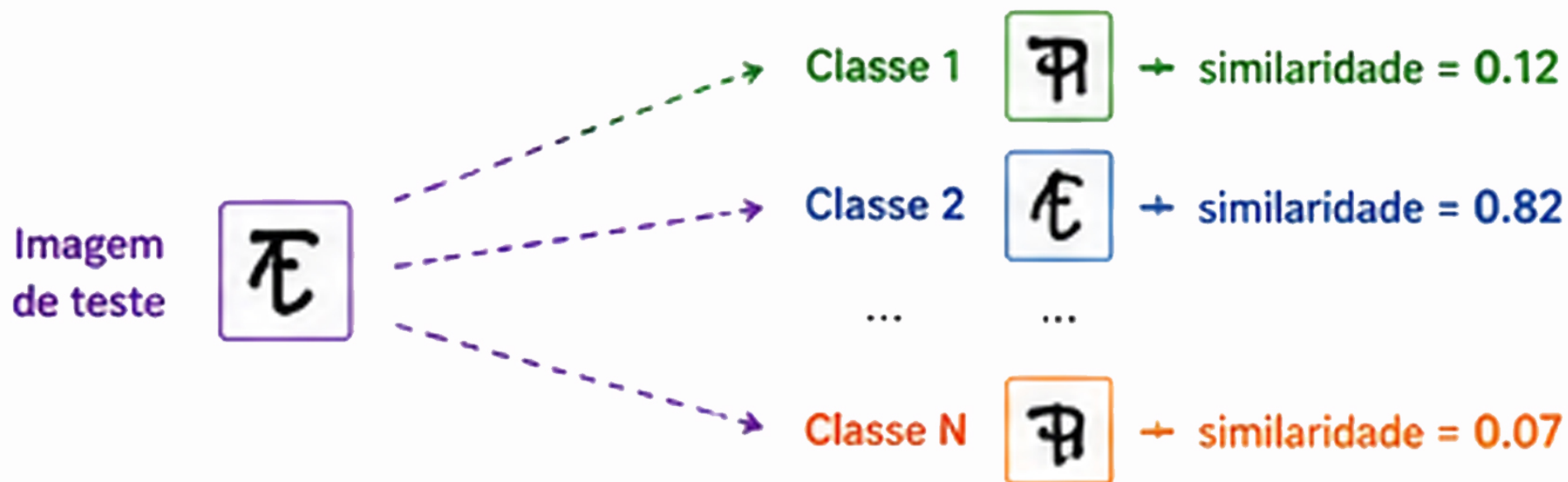
A rede aprende uma representação onde imagens similares ficam próximas e diferentes ficam distantes.



7

COMO FUNCIONA O ONE-SHOT LEARNING

Dadas N classes novas (apenas 1 exemplo de cada) e uma imagem de teste, comparamos a imagem de teste com **cada exemplo** e escolhemos a mais semelhante.

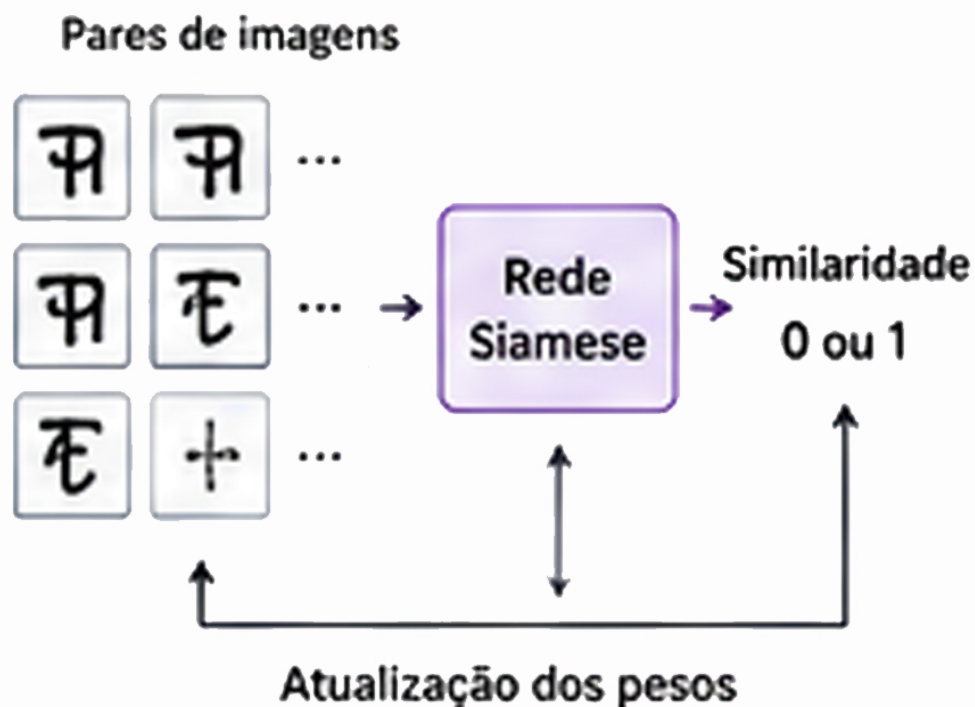


Predição: Classe 2 (maior similaridade)

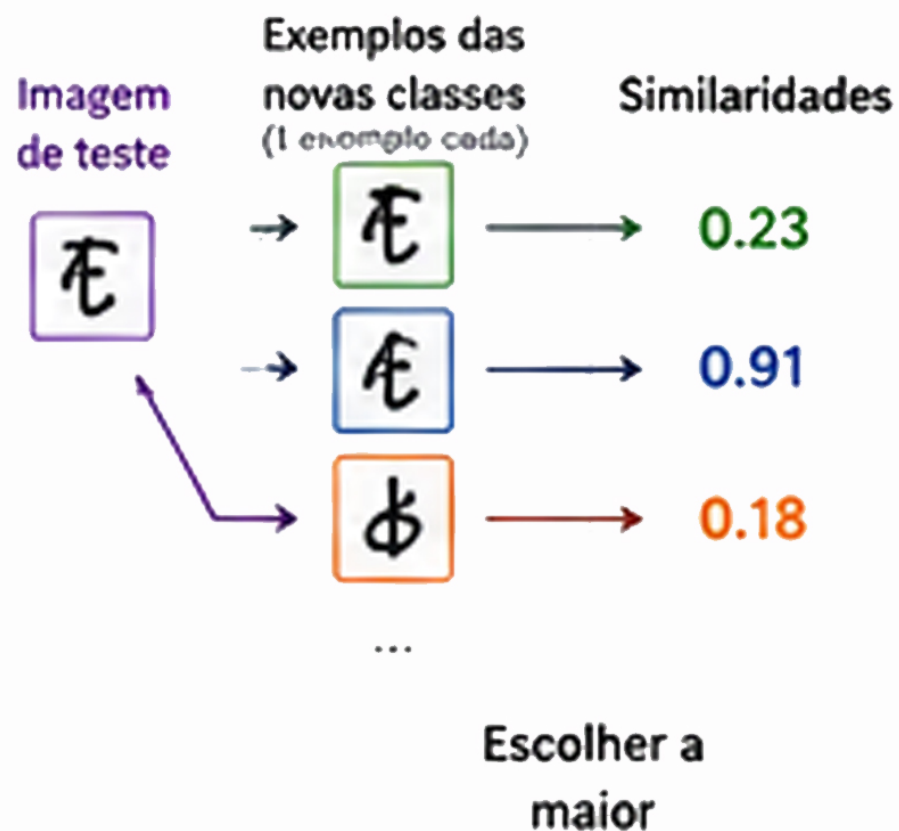
8

FLUXO COMPLETO

FASE DE TREINAMENTO



FASE DE USO (ONE-SHOT)



9

PARA QUE É USADO?

Redes siamesas são úteis em diversas aplicações:



Reconhecimento
facial



Verificação
de identidade



Recuperação
de imagens
(similaridade)



Verificação de
assinaturas



Deteccção de
itens similares



Resumo: Redes siamesas aprendem a comparar duas entradas e medir quão semelhantes elas são. Depois de treinadas, conseguem reconhecer novas classes com apenas um exemplo!



Perguntas?

Referências

[1] Koch, Gregory, Zemel, Richard, Salakhutdinov, Ruslan. Siamese Neural Networks for One-shot Image Recognition. (2015) [*OneShot*](#)

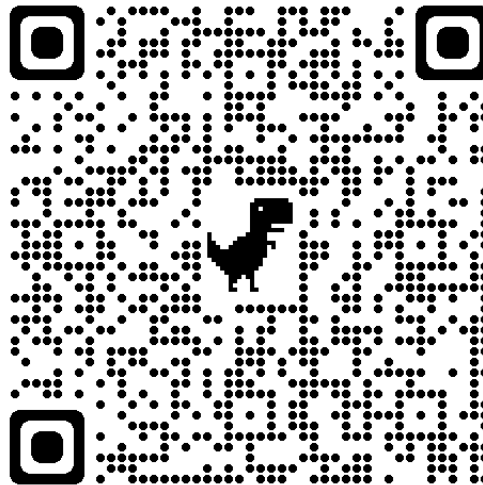
[2] <https://pyimagesearch.com/2020/11/30/siamese-networks-with-keras-tensorflow-and-deep-learning/>

[3] <https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513>

[4] Siamese Neural Networks - <https://youtu.be/T9yKyZfxUJg>

Obrigado!

QUESTIONÁRIO



https://docs.google.com/forms/d/e/1FAIpQLSel_lxLd8P9kd5AZn72LC7pnhS3f-LoAB8aRFRjslt9vQlf2Q/viewform?usp=header

Inatel